

---

(\*.....\*)

MIT License

Copyright (c) [2025] [Angel Montesinos-Amilibia]

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the 'Software.'

*THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS, OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHOR OR COPYRIGHT HOLDER BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.*

(\*.....\*)

## 1. INTRODUCTION.

The program is used to visualize plane curves (initial curves) and other curves derived from them (derivatives).

The user can present them in four ways, namely:

Cartesian Implicit:         $f(x,y)$   
Cartesian Parametric:     $(x(t), y(t))$   
Polar Implicit:             $f(w,z)$   
Polar Parametric:         $(w(t), z(t))$   
                              (w denotes radius, z angle)

These presentations accept up to 4 parameters using the letters **p, q, r, s**.

For example, the presentations could be:

Cartesian Implicit:  
   $f(x,y,parameters): x^3 y + p \cos(y) - q x^2$

Cartesian Parametric:  
   $x(t,parameters): r t + s t^2/(1 + t^2)$   
   $y(t,parameters): t^2 + p$

Polar Implicit:  
   $f(w,z,parameters): w(1 + q \cos(p z)) - r^2$

Polar Parametric:  
   $w(t,parameters): \exp(p t)$   
   $z(t,parameters): t/(1 + q^2)$

The equations of the initial curve can be modified at any

---

---

time, and afterwards a derivative can be chosen, as will be explained further on.

## 2. INITIAL CURVE AND DERIVATIVE CURVE

The program uses two windows: one to set the data of the (initial) input curve, and the (derivative) window to display the drawing of the curves, initial and/or derivative.

Initially, the user determines in the "Presentaciones" menu which of the four they want; writes the formulas of the curves; determines the domain over which the coordinates will range, and sets the parameter values.

For this there are ten "contadores": four for the domains, four for the parameters of the initial curve, and one or two more that appear in some cases (for example, if the derivative curve is the anticaustic of a curve expressed in implicit Cartesian coordinates, a single "contador" appears). Each counter can be operated in two ways:

- 1) Type the number on the left-hand side;
- 2) Press the up or down arrows.

The domains are handled as follows:

Cartesian Implicit:  $f(x, y, \text{parameters})$ :

The program has four counters:

$x_{\text{Min}} < x < x_{\text{Max}},$   
 $y_{\text{Min}} < y < y_{\text{Max}}$   
maximum and minimum of the  $x$  and  $y$  coordinates.

Cartesian Parametric:  $x(t, \text{parameters}), y(t, \text{parameters})$

The program has three counters:

$t_{\text{Min}} < t < t_{\text{Max}},$   
"Scale"  $< 100$   
minimum and maximum of the variable  $t$ , and the scale of the drawing of the initial and derivative curves, if any.

Polar Implícita:  $f(w, z, \text{parameters})$

The interval bounding the curve has three counters  
one counter:

$w > 0$  maximum size of the radius  $w$   
and two counters for the angle  $z$   
 $z_{\text{Min}} < z < z_{\text{Max}}.$

Polar Parametric:  $w(t, \text{parameters}), z(t, \text{parameters})$

The interval bounding the curve has three counters

$t_{\text{Min}} < t < t_{\text{Max}}$   
 $0 < \text{"Scale"} < 100$   
the size of the drawing of the initial curve and/or the derivative.

Once the equations of the initial curve have been written, the "Dibújala" button starts the program to analyze the functions and compute the counter values, so that it can draw the initial curve and the derivative, if there is one (see below).

## 3. DERIVATIVES

we find a list of them, obtained from the initial curves by means of 'well-known' recipes.

---

---

The first one, "None", means that no curve is computed or derived, or that a previously obtained derivative curve is cleared

To explain the derivatives going forward, the following letters will be used:

**J** : the initial curve  
**P** : any point on curve **J**  
**T** : unit tangent at **P**  
**N** : unit normal at **P**  
**G** : the point obtained from **P** by applying the recipe of a derivative  
· : if **A** y **B** are vectors in  $\mathbf{R}^2$ , **A·B** is their dot product.  
**|A|** : is  $\sqrt{\mathbf{A} \cdot \mathbf{A}}$ , where sqrt is the square root.

In general, **P**, **T**, **N**, etc. are understood to be functions of vectors along **J**. For example

**t** --->  $\mathbf{T}(t) = \mathbf{J}'(t)/|\mathbf{J}'(t)|$ , if the curve is parametric,  
**(x,y)** --->  $(-F_y, F_x)/|(-F_y, F_x)|$  if it is implicit, where  $F_y$ ,  $F_x$  denote partial derivatives.

Let us now see how the derivatives are prepared:

"Parallel":

The user is only required to click with the mouse on any point **C** in the window. The program then scans curve **J** looking for the point **P** on **J** closest to **C**. Thus **P-C** will be a vector parallel to **N**, where **N** is the unit normal of **J** at **P**, such that  $\mathbf{C}-\mathbf{P} = r \mathbf{N}$ , i.e.,  $(\mathbf{C}-\mathbf{P}) \cdot \mathbf{N} = r \mathbf{N} \cdot \mathbf{N} = r$ . For any poitn **Q** of **J** with unit normal **M**, the program computes:  $\mathbf{G} = \mathbf{Q} + r \mathbf{M}$ .

"Evolute":

If  $c \neq 0$  is the curvature of **J** in **P**, then the derivative is  $\mathbf{G} = \mathbf{P} + \mathbf{N}/c$ , that is, the derivative is the locus of the centers of curvature (centers of rotation).

"Inversion":

The user clicks the mouse at a point **C** in the window and drags it to another point **A**. The program then draws the circle centered at **C** and radius  $r = |\mathbf{C} - \mathbf{A}|$ , and computes the inversion of **J** relative to that circle. For this, if **P** is any point of **J**, we set  $\mathbf{G} = \mathbf{P} + (r^2/|\mathbf{P}-\mathbf{C}|r^2) (\mathbf{P} - \mathbf{C})$ .

"Involute":

Given a curve **J(t)** with unit tangent **T(t)**, the "Involute" is  $\mathbf{G}(t) = \mathbf{J}(t) - s(t) \mathbf{T}(t)$ , whare **s(t)** is the rc length of **J** from **J(0)** to **J(t)**. For example, if **J** is a circle of radius **r**, we have  $\mathbf{G}(t) = r(\cos(t) + t \sin(t), \sin(t) - t \cos(t))$ .

"Radial":

As Evolute, but now  $\mathbf{G} = \mathbf{Q} + \mathbf{N}/c$ , where **Q** is a fixed point marked by the user with the mouse.

---

---

"Pedal (tangent)":

The user clicks the mouse at some point in the window,  $C$ , which will remain fixed. Let  $P$  be a point of  $J$  whose tangent line is  $t \rightarrow P + t T$ . If  $C$  does not lie on that line, there will be a single line  $u \rightarrow C + u N$  that intersects the other line at exactly one point, i.e., there will be a pair  $(t, u)$  such that

$$P + t T = C + u N.$$

Therefore

$$N \cdot (P + t T) - N \cdot (C + u N) = N \cdot P - N \cdot C - u = N \cdot (P - C) - u = 0,$$

$$T \cdot (P + u T) - T \cdot (C + u N) = T \cdot (P - C) + t = 0$$

from which we get

$$u = N \cdot (P - C),$$

$$t = -T \cdot (P - C),$$

$$G = P + t T = P + T \cdot (C - P) T.$$

"Pedal (normal)":

Follows a similar idea to the previous one, except that instead of using the tangent at point  $P$ , now the unit normal  $N$  is used instead, i.e.,

$$G = P + N \cdot (C - P) N.$$

"Caustic (with parallel illumination)":

The user fixes a segment  $D$  by dragging the mouse between two points in the window. The initial curve is then treated as a mirror illuminated by uniform light parallel to vector  $D$ .

If the mirror is flat, the light rays do not collide with each other, but if it is curved, the light rays will spread apart or collide with one another. The caustic consists of the set of points where these collisions occur.

"Caustic (with local focus)":

The user fixes, with the mouse, a single point  $F$  which will act as the light source, illuminating in all directions with equal intensity. If the mirror is flat, the rays from  $F$  leave it and pass through the image of  $F$ , so that the caustic is a single point. If it is curved, the rays of  $F$  collide with one another and generate the points of the caustic.

"Conchoid":

The user fixes, with the mouse, a point  $C$  in the window, and afterwards fixes a number,  $k$ , in counter  $a$ . After if  $P$  is a point of  $J$ , the following two points are added:

$$P + k (P - C) / |P - C|$$

$$P - k (P - C) / |P - C|.$$

"Strophoid":

The user fixes two points,  $C$  and  $B$ , with the mouse. Then, if  $P$  is a point of  $J$ , it computes the distance between  $P$  and  $B$ :

$$k = |P - B|$$

and adds the following two points:

$$P + k (P - C) / |P - C|$$

$$P - k (P - C) / |P - C|.$$

"Roulette":

Consists of a circle of radius  $a$  that rolls along the initial curve, such that its center remains at a constant distance  $C$  from it.

---

---

"Anticaustic":

An initial curve **J**, a fixed point **C** in  $\mathbb{R}^2$ , and a fixed scalar  $m > 0$ : If **P** is a point of **J**, and **T** is its unit tangent at **P** such that  $|\mathbf{C} - \mathbf{P}| < m$ , one must find the points  $\mathbf{G} = \mathbf{P} + s \mathbf{T}$  on the tangent line such that its distance to **C** plus its distance to **P** equals  $m$ . Since  $\mathbf{T} \cdot \mathbf{T} = 1$ , the distance from **P** to **G** will equal  $|s|$ , so that  $|\mathbf{C} - \mathbf{G}| + |s| = m$  and a simple calculation gives two solutions for  $s$ :

$$s_1 = (m^2 - |\mathbf{C} - \mathbf{P}|^2) / (2 (m - \mathbf{T} \cdot (\mathbf{C} - \mathbf{P}))),$$
$$s_2 = -(m^2 - |\mathbf{C} - \mathbf{P}|^2) / (2 (m + \mathbf{T} \cdot (\mathbf{C} - \mathbf{P}))),$$

leading to the two solutions  $\mathbf{G}_1 = \mathbf{P} + s_1 \mathbf{T}$ ,  $\mathbf{G}_2 = \mathbf{P} + s_2 \mathbf{T}$ .

"Reflection":

The user fixes a point **C** in the window, giving us, for any point **P** of **J**, the following

$$t = (\mathbf{P} - \mathbf{C}) \cdot \mathbf{T},$$
$$\mathbf{G} = \mathbf{C} + 2 t \mathbf{T}.$$

"Refraction":

The user fixes a point **C** in the window, giving us, for any point **P** of **J**, the following

$$t = (\mathbf{P} - \mathbf{C}) \cdot \mathbf{N}$$
$$\mathbf{G} = \mathbf{C} + 2 t \mathbf{T}.$$

#### 4. EXAMPLES

A collection of well-known curves is included, to which some parameters have been added so that users can play around with them as they wish.

#### 5. HOW TO WRITE THE FORMULAS

In the data input window, when for example "Cartesiana Implícita" has been selected, an editable field titled appears

**F(x,y,parameters),**

You must write functions in the variables that appear, that is **x, y**, for the variable coordinates, and also any of the parameters

**p, q, r, s.**

Example:

$$x^3 y - p x^2 - q y^2 + r^2 \sin(3 y).$$

The program is quite flexible with regard to how formulas are written. That is, it recognizes different names for some functions, for example "arctan" = "atan"; it does not require the symbol \* for multiplication, and often does not even require a blank space; however, to be on the safe side, a blank space between factors is sufficient. It recognizes the word

pi como **3.141592...** (but not the Greek letter).

---

---

On the other hand, it requires that function arguments be enclosed in parentheses, and that numbers be written as 43.23, -0.146, etc. That is, you should not use the format .87 nor 0.13e-12. The decimal separator must be the point.

In addition to the operations:

- + (plus),
- (minus),
- \* (times),
- / (divided by), and
- $a^b$  or  $a**b$  (a raised to the power b),

the following functions can be used (in parentheses, and after an equals sign, are listed the different names the program recognizes):

- sqr (square), sqrt (square root), sin (= sen),
- cos, tan (= tg), cot (= cotan = cotg), arcsin (= asin
- = arcsen = asen), arccos (= acos = arcos), arctan
- (= atan = arctg), exp, ln (= log), sinh (= sh),
- cosh (= ch), tanh (= th), arcsinh (= asinh = argsh),
- arccosh (= acosh = argch), arctanh (= atanh =
- argtanh = argth), abs (absolute value).

(\*.....\*)

Copyright Angel Montesinos-Amilibia, 2026  
Angel Montesinos-Amilibia,  
(España),  
09/01/2026'  
e-mail: montesin@uv.es'

---